SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
Off Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I ARUNPRASATH R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
ARUNPRASATH R

Annexure A

DESIGN TASK

S.No	Register Number	Questions
3	192524077	Design a parser to recognize Boolean expressions. **Grammar** B → B && B B → B || B B → !B B → (B) B → true B → false **Input** true && false || true **Design Requirements** (i) Implement the lexical analyzer using Lex. (ii) Implement the parser. (iii) Validate the Boolean expression. (iv) Display Accepted or Rejected. (v) Handle syntax errors.

		Design a parser to recognize variable declarations. **Grammar** D → Type IdList ; Type → int | float | char IdList → id IdList → id , IdList **Input** int a, b, c; **Design Requirements** (i) Implement the lexical analyzer using Lex. (ii) Implement the parser. (iii) Validate the declaration. (iv) Display Accepted or Rejected.

		(v) Handle syntax errors.
		Design a parser to recognize assignment statements. **Grammar** S → id = E ; E → E + T E → T T → id T → num **Input** x = a + 10; **Design Requirements** (i) Implement the lexical analyzer using Lex. (ii) Implement the parser. (iii) Validate the assignment statement. (iv) Display Accepted or Rejected. (v) Handle syntax errors.

		Design a parser to recognize function declarations.
		
		Grammar
		
		F → Type id ( ) Block
		
		Type → int | float | void
		
		Block → { }
		
		Input
		
		int display()
		
		{
		
		}
		
		Design Requirements
		
		(i) Implement the lexical analyzer using Lex.
		
		(ii) Implement the parser.
		
		(iii) Validate the function declaration.
		
		(iv) Display Accepted or Rejected.
		
		(v) Handle syntax errors.

		Design a parser to recognize function calls. **Grammar** Call → id (ArgList) ArgList → id
		ArgList → id , ArgList ArgList → ε **Input** sum(a,b,c) **Design Requirements** (i) Implement the lexical analyzer using Lex. (ii) Implement the parser. (iii) Validate the function call. (iv) Display Accepted or Rejected. (v) Handle syntax errors.

1.

```
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> bison -dy variable.y.txt
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> flex variable.l.txt
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> gcc y.tab.c lex.yy.c -o variable.exe
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> .\variable.exe
Enter Variable Declaration:
int a;
^Z
Accepted
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB>
```

2.

```
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> flex boolean.l
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> gcc y.tab.c lex.yy.c -o boolean.
exe
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> .\boolean.exe
Enter Boolean Expression:
true && false || true

^Z
Accepted
```

3.

```
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> bison -dy assignment.y.txt
conflicts: 16 shift/reduce
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB>
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> flex assignment.l.txt
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> gcc y.tab.c lex.yy.c -o assignm
ent.exe
PS C:\Users\arunp\OneDrive\Documents\Compiler - LAB> .\assignment.exe
Enter Assignment Statement: a = 10;
Valid Assignment
```

4.

```
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>set PATH=C:\GnuWin32\bin;C:
\MinGW\bin;%PATH%

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>flex funcdecl.l.txt

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>bison -dy funcdecl.y.txt

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc lex.yy.c y.tab.c -o fun
cdecl.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>funcdecl.exe
'C:\Users\arunp\OneDrive\Documents\Compiler - LAB\funcdecl.exe' was blocked
by your organization's Device Guard policy.
Contact your support person for more info.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc lex.yy.c y.tab.c -o a.e
xe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>a.exe
Enter Function Declaration:
int display()
{
}

Accepted
```

5.

```
(c) Microsoft Corporation. All rights reserved.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>set PATH=C:\GnuWin32\bin;C:\MinGW\bin;%PATH%

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>flex funccall.l.txt

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>bison -dy funccall.y.txt

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc lex.yy.c y.tab.c -o funccall.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>funccall.exe
'C:\Users\arunp\OneDrive\Documents\Compiler - LAB\funccall.exe' was blocked by your organization's Device Guard policy.
Contact your support person for more info.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>a.exe
Enter Function Declaration:
sum(a,b,c)

Rejected

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc lex.yy.c y.tab.c -o a.exe
c:/mingw/bin/../lib/gcc/mingw32/6.3.0/../../../../mingw32/bin/ld.exe: cannot open output file a.exe: Permission denied
collect2.exe: error: ld returned 1 exit status

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>taskkill /F /IM a.exe
SUCCESS: The process "a.exe" with PID 896 has been terminated.

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>gcc lex.yy.c y.tab.c -o a.exe

C:\Users\arunp\OneDrive\Documents\Compiler - LAB>a.exe
Enter Function Call:
sum(a,b,c)

Accepted
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrat es thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Welldesigned solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate constructio n of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis

Documentation & Presentation	10	Wellstructured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation